

Embedded
Online
Conference

BENINGO
EMBEDDED GROUP



Getting Started with Zephyr RTOS



Jacob Beningo



Mohammed Billoo

THE SPEAKER



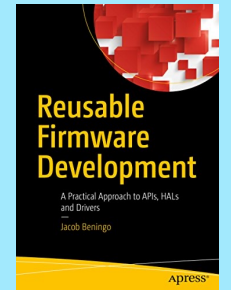
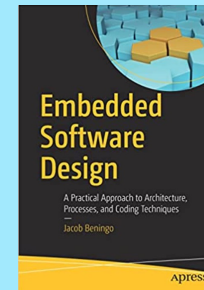
Jacob Beningo



Beningo Embedded Group CEO & Founder

Focus: Embedded Software Consulting and Training

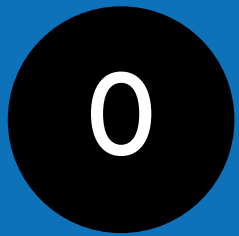
Specializes in the design of real-time, microcontroller based embedded software.



Blogs for Embedded.com, DesignNews.com, etc, focused on embedded system design techniques and challenges.

Visit www.beningo.com to learn more ...





Session Goals

“Without a defined path, every path is aimless”

- Unknown

AGENDA

1

Introducing The Zephyr Project

2

Zephyr RTOS Fundamentals

3

Zephyr Application Model

4

“Hello World!” Example

5

Advanced Topics

6

Going Further

1

Introducing the Zephyr Project

Introducing the Zephyr Project

An open-source RTOS (Apache 2.0)

- Linux Foundation supported
- Built with safety and security in mind
- Lightweight kernel with supporting services
 - Connectivity stacks like Wi-Fi, BLE, USB, MQTT, etc
- Developer Friendly
 - Logging, tracing, built-in-shell, Windows/Linux/macOS
- Ships with SBOMs (Software Bill of Materials)
- Supports more than [500 boards](#)



BENINGO
EMBEDDED GROUP

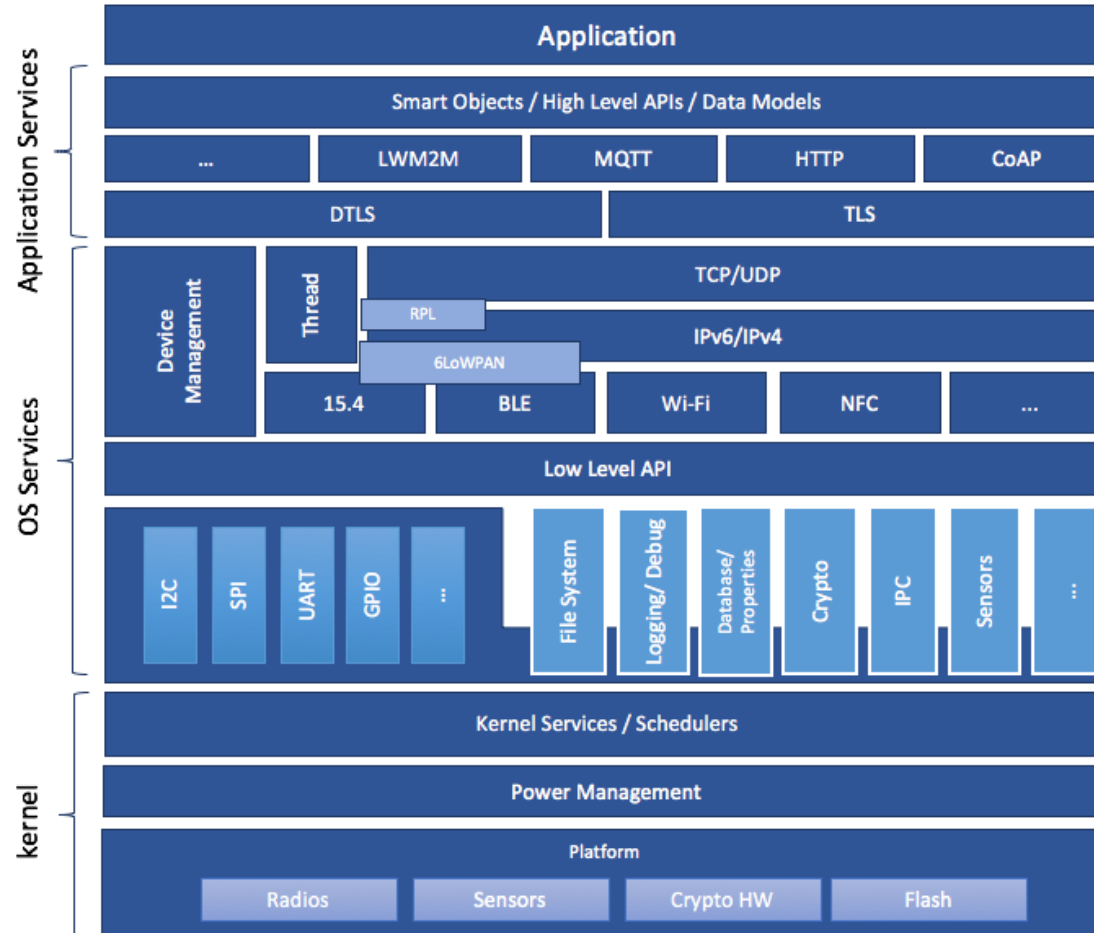
BENINGO
EMBEDDED GROUP

Additional Resources: [Zephyr Project Overview Document](#)

Introducing the Zephyr Project

Zephyr RTOS Architecture

BENINGO
EMBEDDED GROUP



BENINGO
EMBEDDED GROUP

Introducing the Zephyr Project

Zephyr APIs

BENINGO
EMBEDDED GROUP

API Overview

The table lists Zephyr's APIs and information about them, including their current [stability level](#). More details about API changes between major releases are available in the [Releases](#).

API	Status	Version Introduced
Analog-to-Digital Converter (ADC)	Stable	1.0
Audio Codec	Experimental	1.13
Digital Microphone (DMIC)	Experimental	1.13
Auxiliary Display (auxdisplay)	Experimental	3.4
Barriers API	Experimental	3.4
Bootloader Information	Experimental	3.5
Bluetooth APIs	Stable	1.0
Clock Control	Stable	1.0
CoAP	Unstable	1.10
Connection Manager	Experimental	3.4.0
CAN Controller	Stable	1.14
CAN Transceiver	Experimental	3.1

BENINGO
EMBEDDED GROUP

Introducing the Zephyr Project

POSIX API's

POSIX (Portable Operating System Interface for Unix) is a set of standards that define the API (Application Programming Interface) for Unix-like operating systems

API Set	Description / Use
pthread_create()	Create a thread
pthread_mutex_lock()	Lock a shared resource
pthread_cond_wait()	Wait on a condition
sem_post()	Give a semaphore token
open()	Open a file
etc	Other POSIX APIs

BENINGO
EMBEDDED GROUP

BENINGO
EMBEDDED GROUP

Introducing the Zephyr Project

Zephyr Kernel APIs

BENINGO
EMBEDDED GROUP

API Set	Description / Use
Threads	Creating, scheduling, and deleting independent executable threads of instructions
Scheduling	Idle, sleep, yield
Workqueue Threads	Uses a dedicated thread to process work items in a first in, first out manner
Interrupts	Multi-level interrupt handling
Semaphores	Implements a traditional counting semaphore.
Mutexes	Implements a traditional reentrant mutex
Events	Implements traditional events

BENINGO
EMBEDDED GROUP

2

Zephyr RTOS Fundamentals

Zephyr RTOS Fundamentals

Tasks, Threads, and Processes

BENINGO
EMBEDDED GROUP

Definition:

- [1] A **task** is a concurrent and independent program that competes for execution time on a CPU.
- [2] a semi-independent portion of the application that carries out a specific duty.

Definition:

- [1] A **thread** is a semi-independent program segment that executes within a process.

Definition:

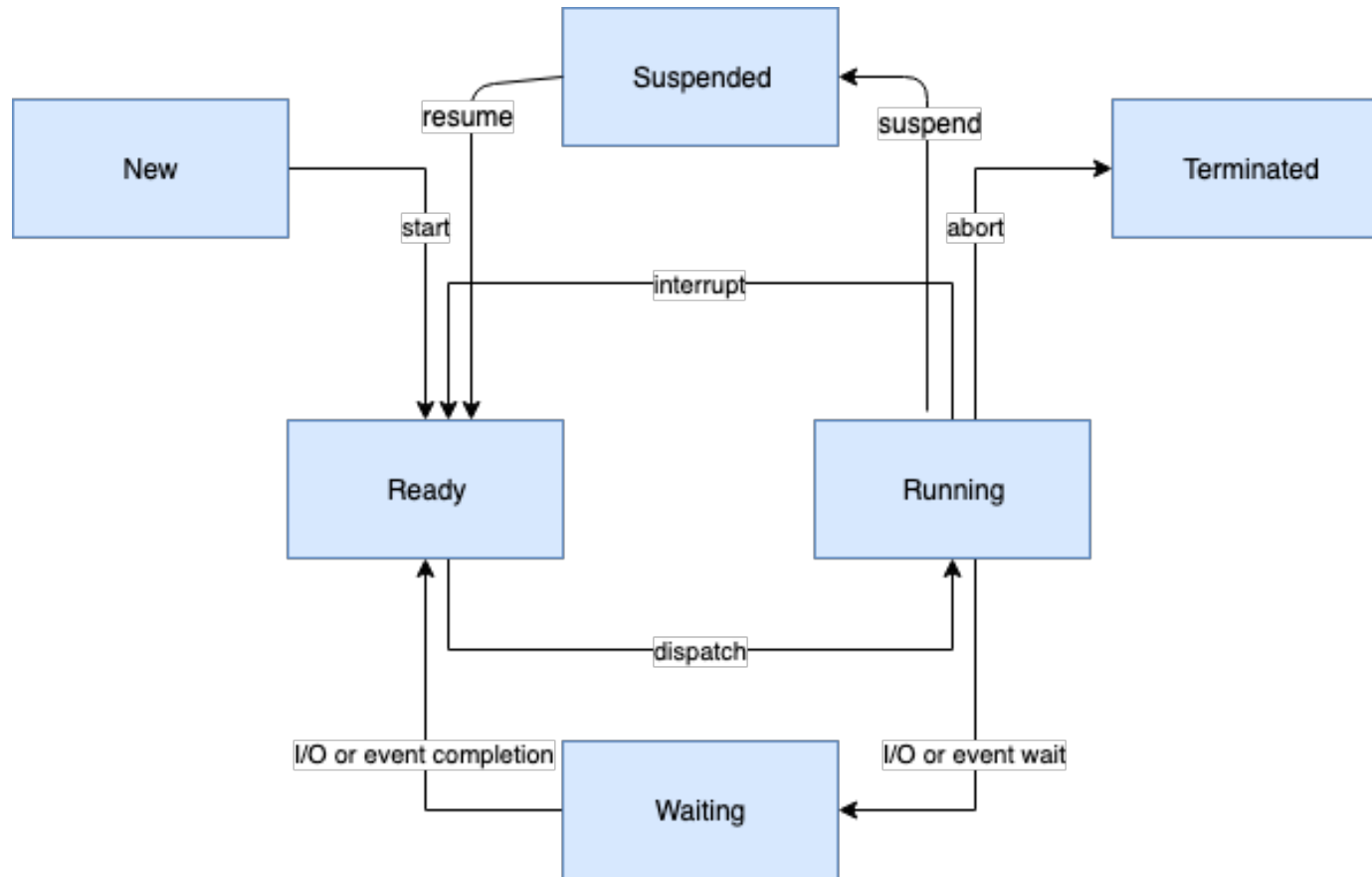
- [1] A **process** is a collection of threads and associated memory that run in an independent memory location.

BENINGO
EMBEDDED GROUP

Zephyr RTOS Fundamentals

Thread States

BENINGO
EMBEDDED GROUP



BENINGO
EMBEDDED GROUP

Zephyr RTOS Fundamentals

Thread Creation

BENINGO
EMBEDDED GROUP

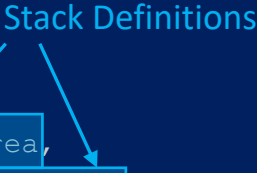
```
#define MY_STACK_SIZE (500U)
#define MY_PRIORITY (5U)

extern void Thread_LedBlinky(void *, void *, void *);

K_THREAD_STACK_DEFINE(my_stack_area, MY_STACK_SIZE);

struct k_thread my_thread_data;

k_tid_t my_tid = k_thread_create(&my_thread_data, my_stack_area,
                                K_THREAD_STACK_SIZEOF my_stack_area,
                                my_entry_point, NULL, NULL, NULL,
                                MY_PRIORITY, 0, K_NO_WAIT);
```



BENINGO
EMBEDDED GROUP

Zephyr RTOS Fundamentals

Thread Creation

K_THREAD_DEFINE(name, stack_size, entry, p1, p2, p3, prio, options, delay)

```
#define MY_STACK_SIZE (500U)
#define MY_PRIORITY (5U)

extern void Thread_LedBlinky(void *, void *, void *);

K_THREAD_DEFINE(my_tid,           // Name of the thread (not dev defined!)
                MY_STACK_SIZE,    // Stack size in bytes
                Thread_LedBlinky, // Thread Entry Function
                NULL, NULL, NULL, // Custom parameters
                MY_PRIORITY,       // Thread Priority
                0,                 // Thread Options
                0);               // Start scheduling delay
```

BENINGO
EMBEDDED GROUP

BENINGO
EMBEDDED GROUP

Zephyr RTOS Fundamentals

Semaphores

BENINGO
EMBEDDED GROUP

A **semaphore** is used to synchronize application behavior between

- An ISR and a task
- One task to another task
- Occasionally are used to protect a shared resource but this is not their primary purpose (Not Recommended)

```
// Define a semaphore
struct k_sem my_sem;
k_sem_init(&my_sem, 0, 10);

// Define a semaphore with macro
K_SEM_DEFINE(my_sem, 0, 1);

// Semaphore Operations
k_sem_give(&my_sem);
k_sem_take(&my_sem, K_MSEC(50))
```

```
void consumer_thread(void){
    ...

    if (k_sem_take(&my_sem, K_MSEC(50)) != 0) {
        printk("Input data not available!");
    } else { . . .
    }

    ...
}
```

BENINGO
EMBEDDED GROUP

Zephyr RTOS Fundamentals

Mutexes

A **mutex** is used to gain access to a shared resource such as

- A memory location
- A common peripheral like a UART

```
// Define a mutex
struct k_mutex my_mutex;
k_mutex_init(&my_mutex);

// Define a mutex with a macro
K_MUTEX_DEFINE(my_mutex);

// Mutex Operations
k_mutex_lock(&my_mutex, K_FOREVER);
k_mutex_unlock(&my_mutex);
```

```
void consumer_thread(void) {
    ...

    if (k_mutex_lock(&my_mutex, K_MSEC(50)) != 0) {
        printk("Accessing Shared Data!");
    } else {
        printk("Cannot lock Shared Data!");
    }

    ...
}
```

BENINGO
EMBEDDED GROUP

BENINGO
EMBEDDED GROUP



3

Introducing The Zephyr Project RTOS

THE SPEAKER

mab@mab-labs.com



 /mab-embedded

 @mabembedded



- Embedded Software Consultant
- Design Work
 - Medical Devices
 - Scientific Instruments
 - LIDAR
 - Custom ASIC
 - FPGA
- Expertise
 - Zephyr
 - Yocto/Embedded Linux
 - Qt

The Zephyr Project RTOS

Self-contained

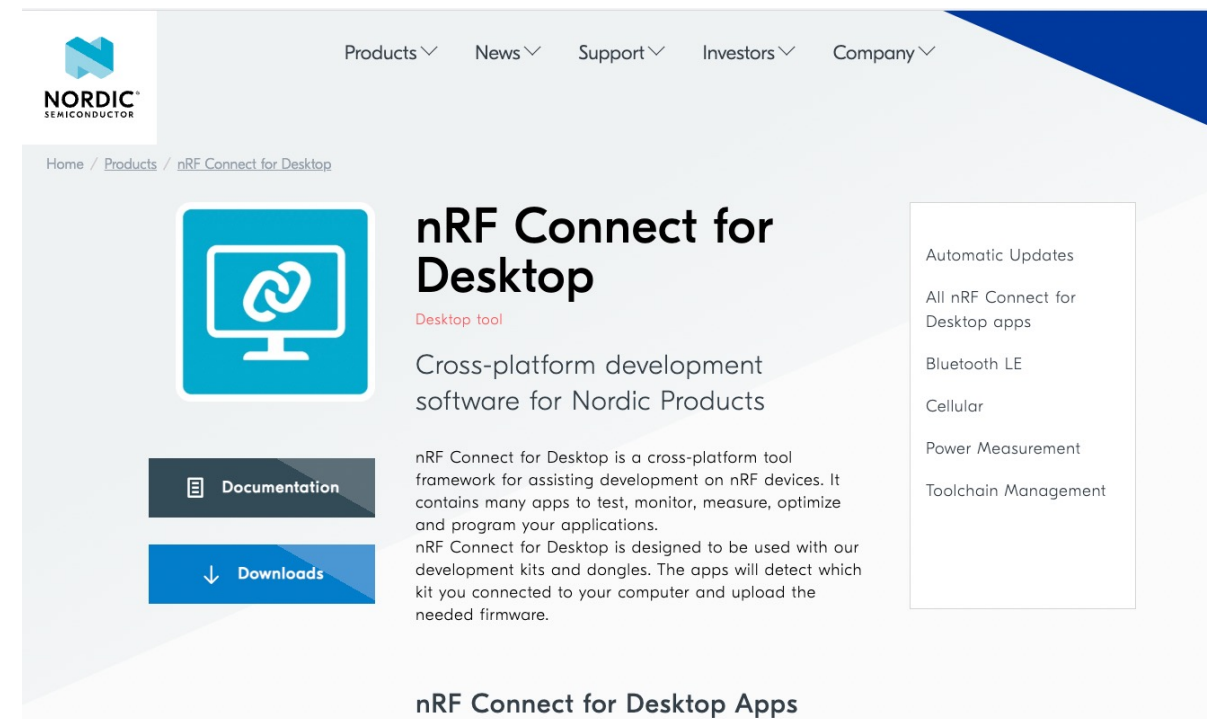
- Use “West” to download *everything*
- Support for:
 - CPUs
 - Boards
 - Bootloader
 - Subsystems
 - **Peripherals**
- Drivers are **vett**ed
 - Thriving open-source community with maintainers

Getting Started

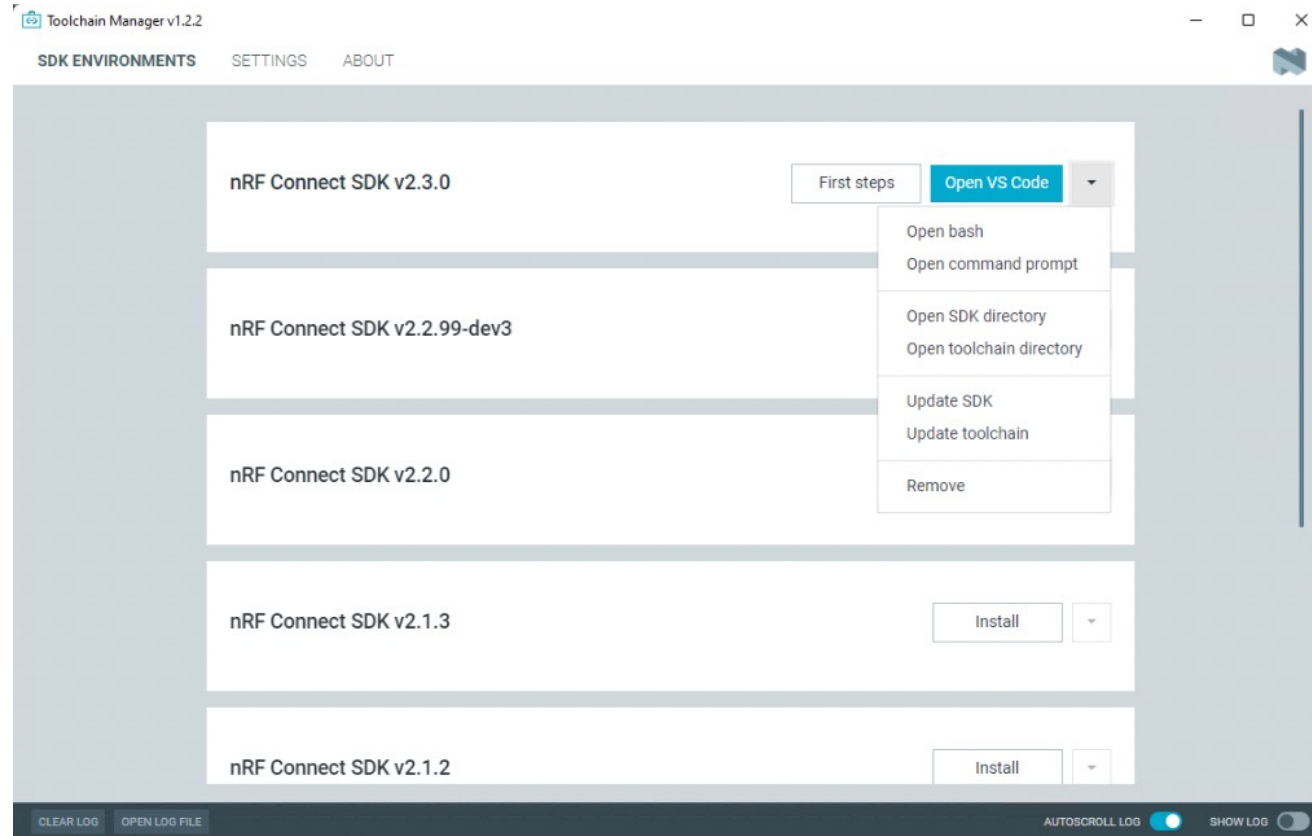
nRF Connect For Desktop

- An application that sets up a complete Zephyr development environment
 - Manages West
- VS Code Plugins!
 - Create new project
 - Based on example
 - Open existing project
 - Configure Zephyr
 - Build and debug project

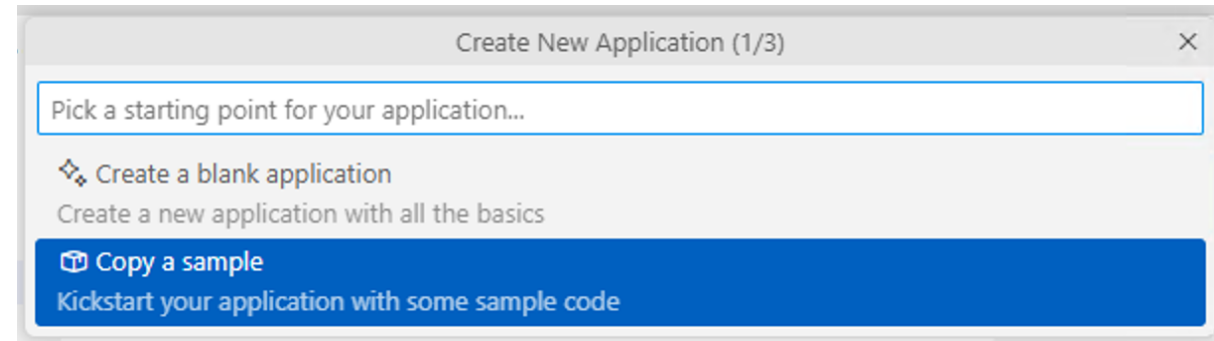
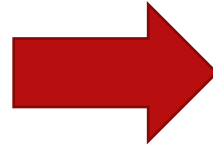
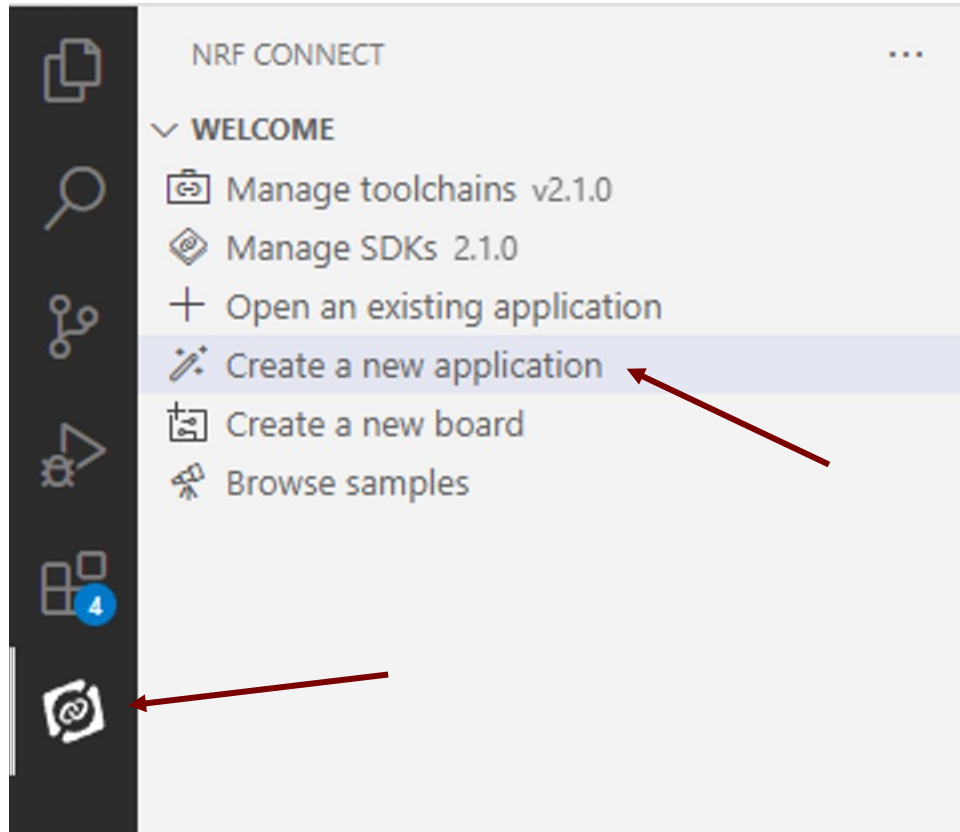
<https://www.nordicsemi.com/Products/Development-tools/nrf-connect-for-desktop>



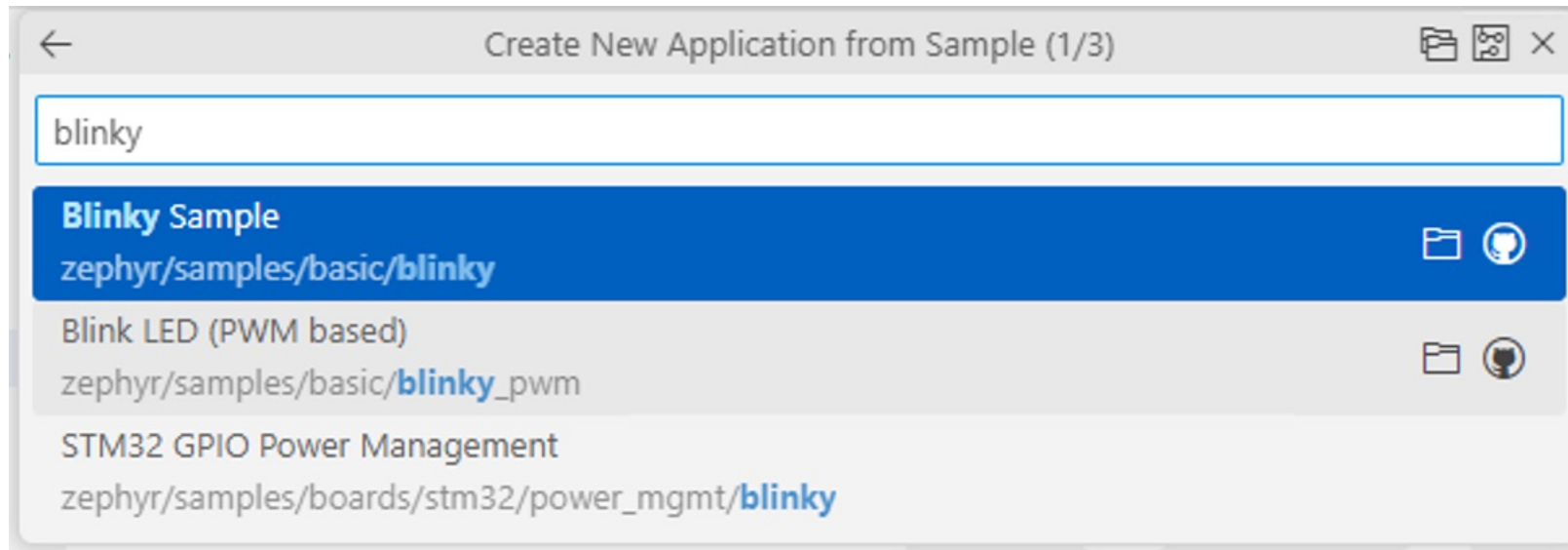
nRF Connect For Desktop



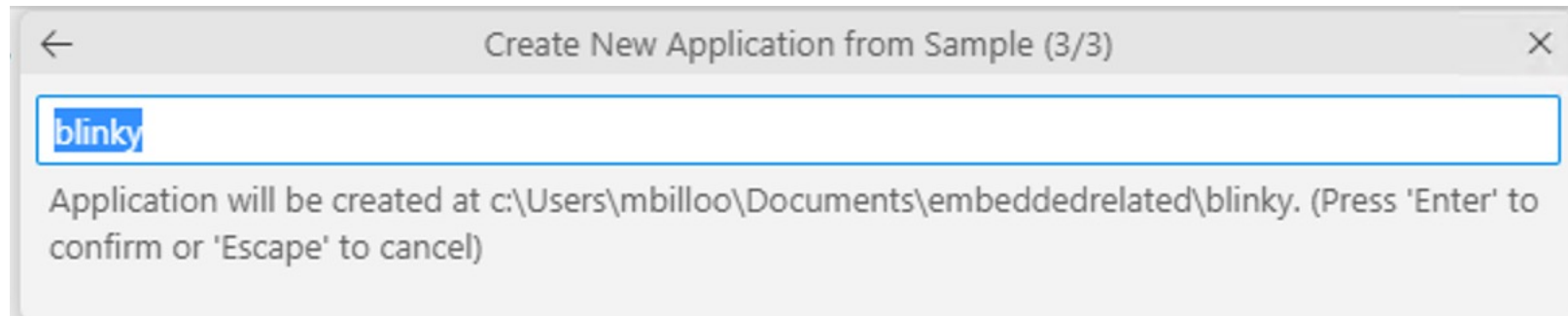
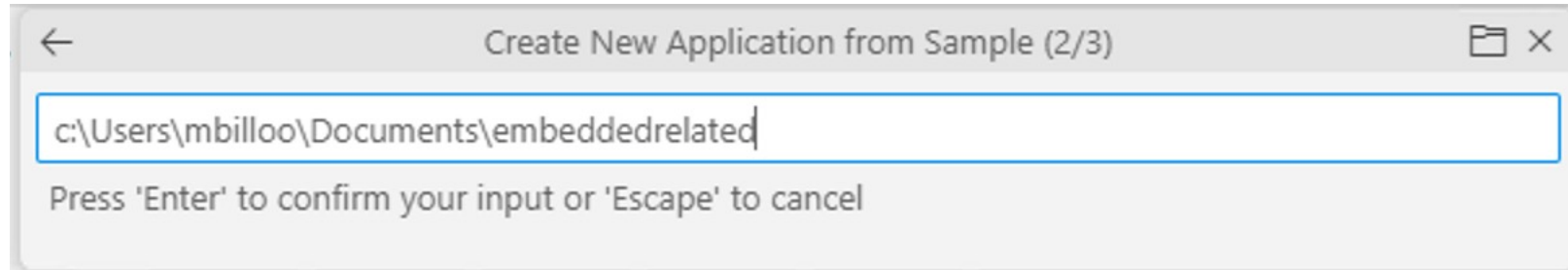
nRF Connect SDK



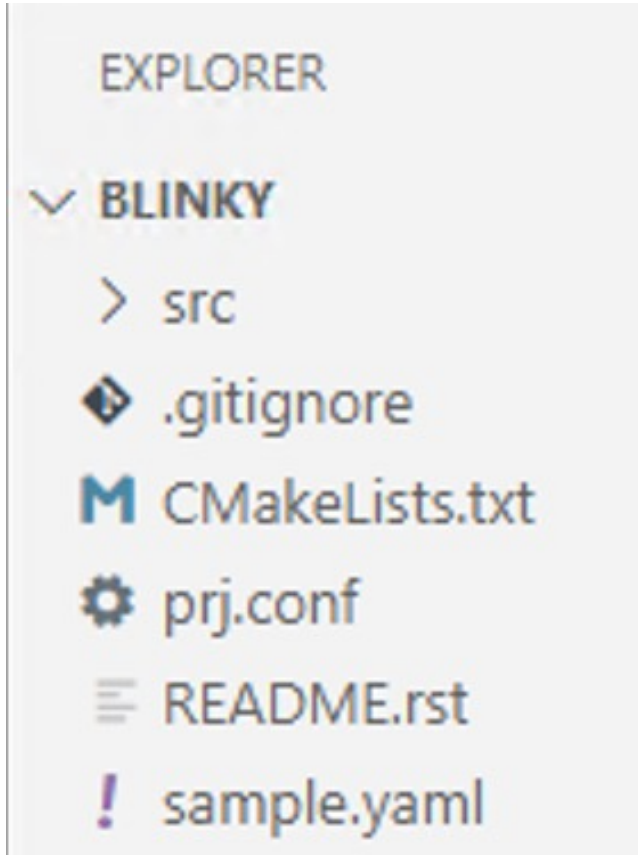
nRF Connect SDK



nRF Connect SDK

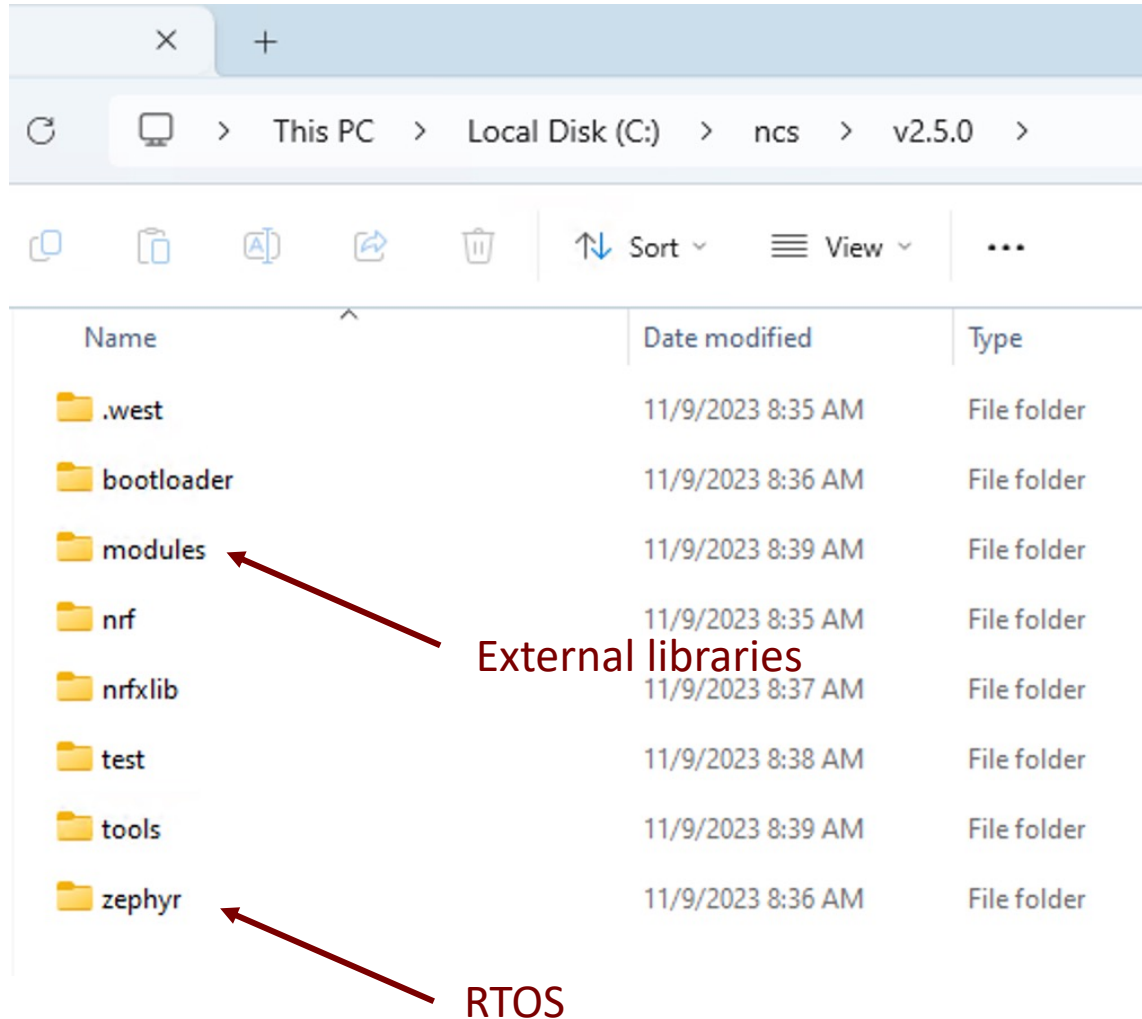


Application Organization



Will review application organization later

Zephyr Organization

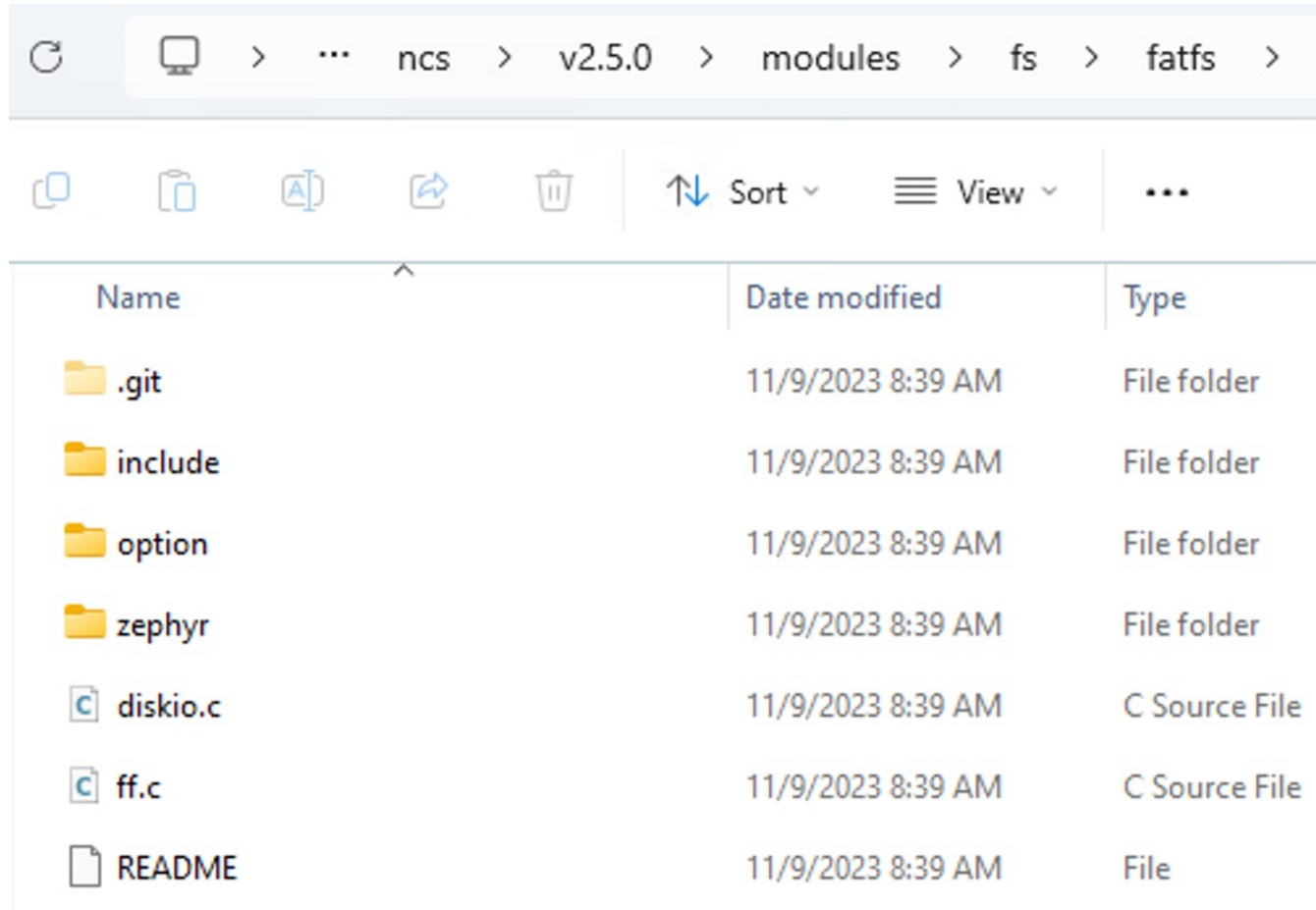


Name	Date modified	Type
.west	11/9/2023 8:35 AM	File folder
bootloader	11/9/2023 8:36 AM	File folder
modules	11/9/2023 8:39 AM	File folder
nrf	11/9/2023 8:35 AM	File folder
nrfxlib	11/9/2023 8:37 AM	File folder
test	11/9/2023 8:38 AM	File folder
tools	11/9/2023 8:39 AM	File folder
zephyr	11/9/2023 8:36 AM	File folder

External libraries

RTOS

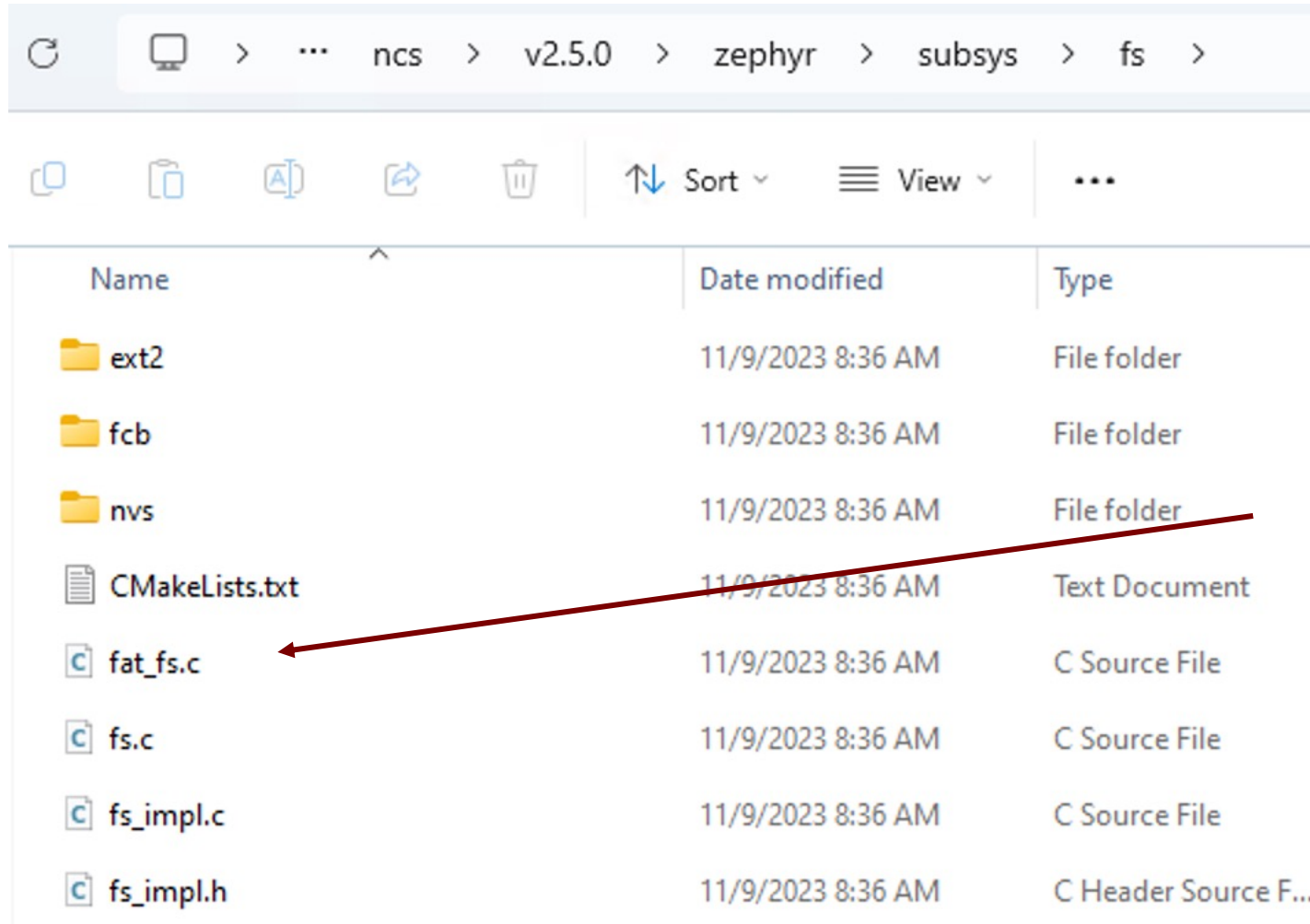
Zephyr Organization



Name	Date modified	Type
.git	11/9/2023 8:39 AM	File folder
include	11/9/2023 8:39 AM	File folder
option	11/9/2023 8:39 AM	File folder
zephyr	11/9/2023 8:39 AM	File folder
diskio.c	11/9/2023 8:39 AM	C Source File
ff.c	11/9/2023 8:39 AM	C Source File
README	11/9/2023 8:39 AM	File

Third party FATFS library
(included with Zephyr)

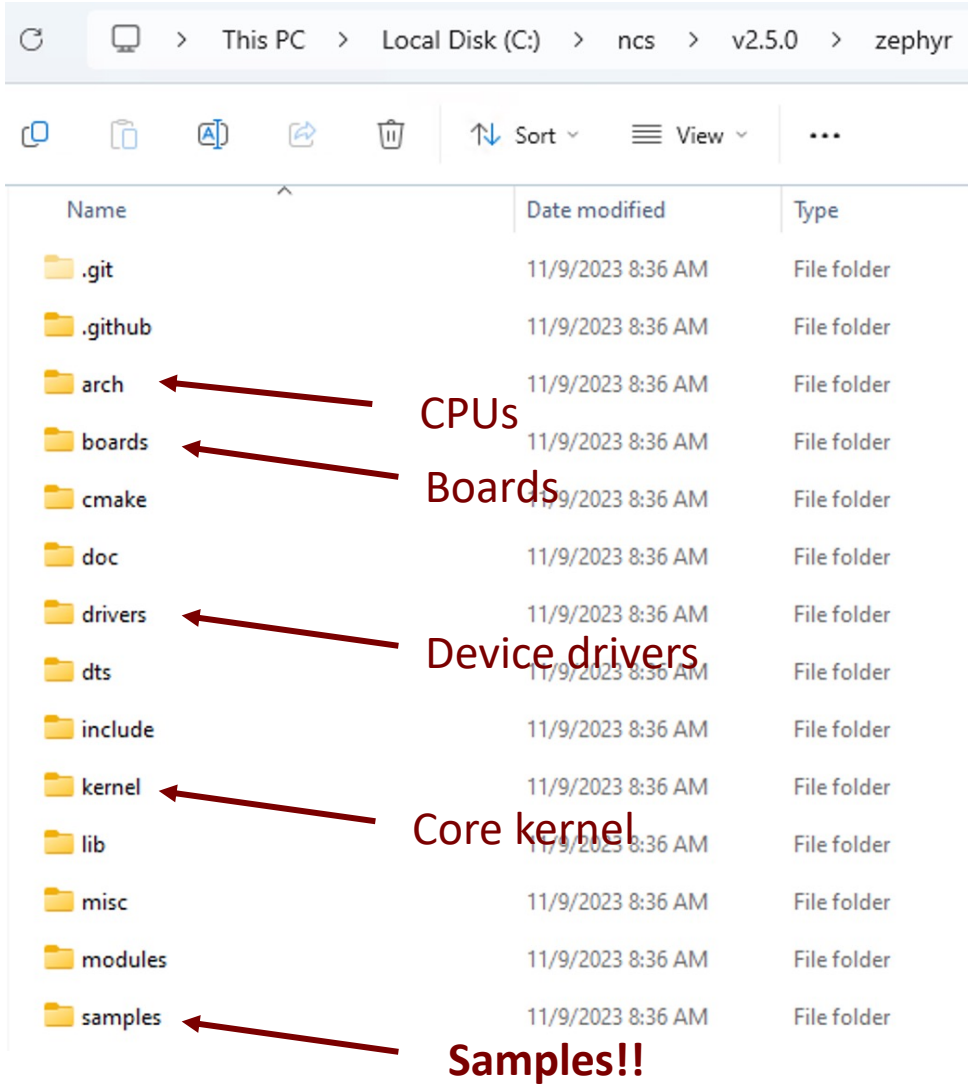
Zephyr Organization



Name	Date modified	Type
ext2	11/9/2023 8:36 AM	File folder
fcb	11/9/2023 8:36 AM	File folder
nvs	11/9/2023 8:36 AM	File folder
CMakeLists.txt	11/9/2023 8:36 AM	Text Document
fat_fs.c	11/9/2023 8:36 AM	C Source File
fs.c	11/9/2023 8:36 AM	C Source File
fs_impl.c	11/9/2023 8:36 AM	C Source File
fs_impl.h	11/9/2023 8:36 AM	C Header Source F...

Interface from Zephyr to
FATFS library

Zephyr Organization

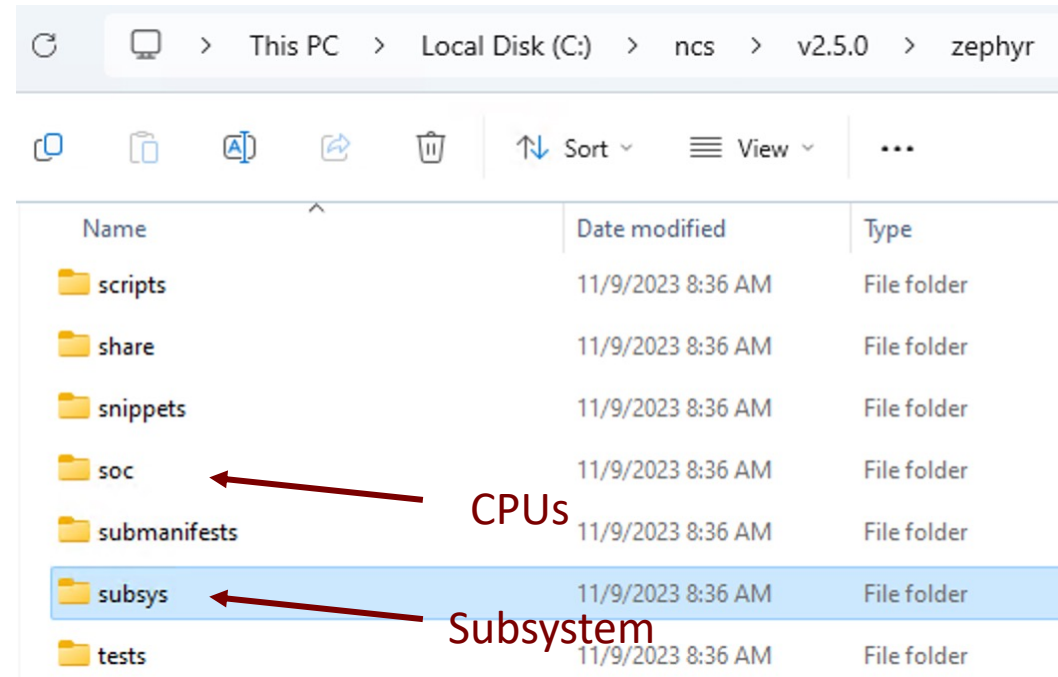


Windows File Explorer view of the Zephyr directory structure. The path is This PC > Local Disk (C:) > ncs > v2.5.0 > zephyr. The table below lists the contents of the directory.

Name	Date modified	Type
.git	11/9/2023 8:36 AM	File folder
.github	11/9/2023 8:36 AM	File folder
arch	11/9/2023 8:36 AM	File folder
boards	11/9/2023 8:36 AM	File folder
cmake	11/9/2023 8:36 AM	File folder
doc	11/9/2023 8:36 AM	File folder
drivers	11/9/2023 8:36 AM	File folder
dts	11/9/2023 8:36 AM	File folder
include	11/9/2023 8:36 AM	File folder
kernel	11/9/2023 8:36 AM	File folder
lib	11/9/2023 8:36 AM	File folder
misc	11/9/2023 8:36 AM	File folder
modules	11/9/2023 8:36 AM	File folder
samples	11/9/2023 8:36 AM	File folder

Annotations:

- CPUs (points to **arch**)
- Boards (points to **boards**)
- Device drivers (points to **drivers**)
- Core kernel (points to **kernel**)
- Samples!! (points to **samples**)



Windows File Explorer view of the subdirectories within the Zephyr directory. The path is This PC > Local Disk (C:) > ncs > v2.5.0 > zephyr. The table below lists the contents of the directory.

Name	Date modified	Type
scripts	11/9/2023 8:36 AM	File folder
share	11/9/2023 8:36 AM	File folder
snippets	11/9/2023 8:36 AM	File folder
soc	11/9/2023 8:36 AM	File folder
submanifests	11/9/2023 8:36 AM	File folder
subsys	11/9/2023 8:36 AM	File folder
tests	11/9/2023 8:36 AM	File folder

Annotations:

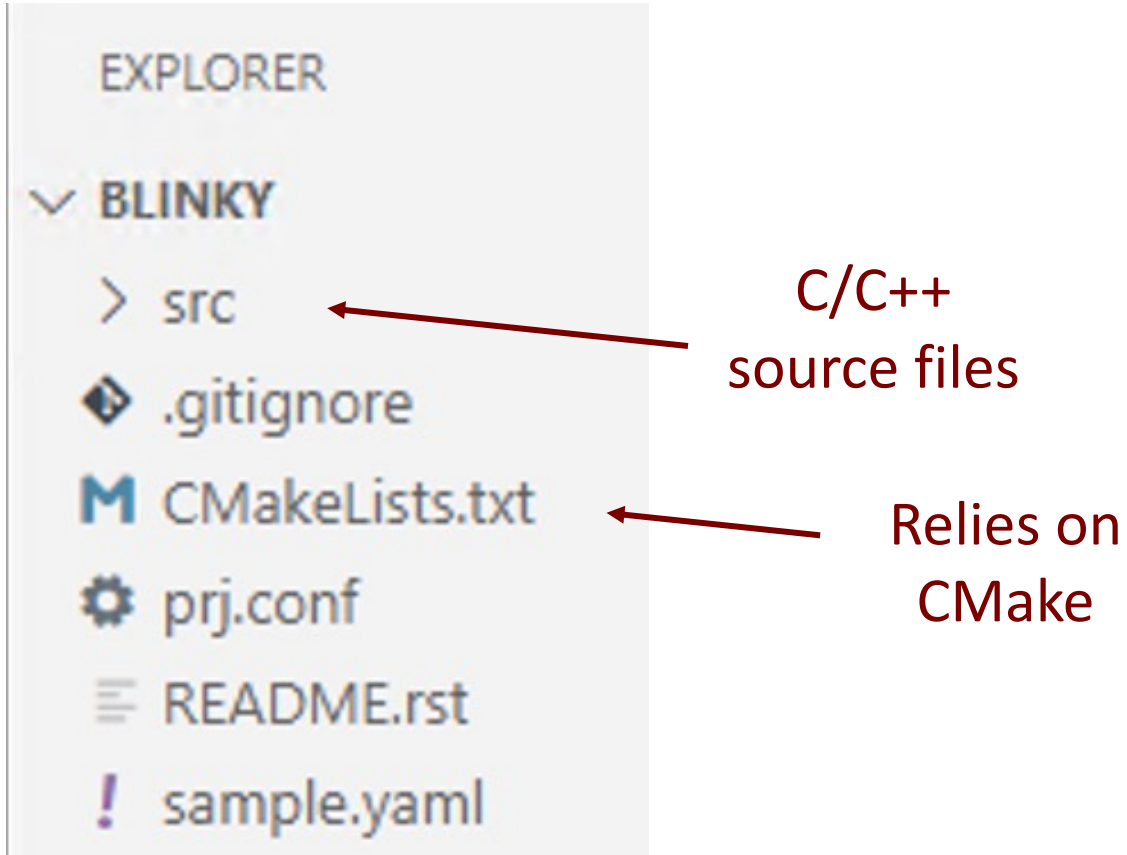
- CPUs (points to **soc**)
- Subsystem (points to **subsys**)



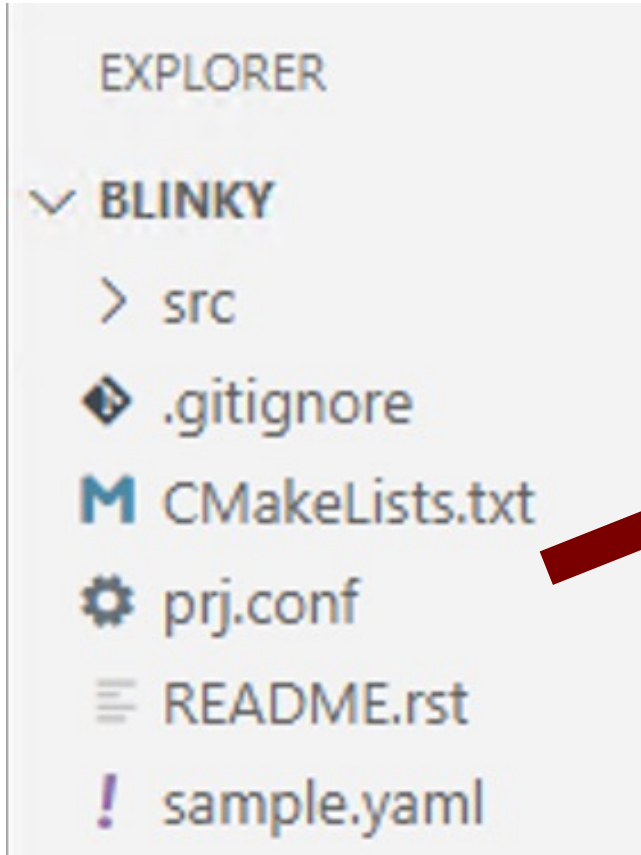
4

Zephyr Application Model

Key Elements



Project Configuration



Enables the GPIO
subsystem in Zephyr

Project Configuration

```
[app 09:38:57]$ cat prj.conf
CONFIG_DISK_ACCESS=y
CONFIG_LOG=y
CONFIG_FILE_SYSTEM=y
CONFIG_FAT_FILESYSTEM_ELM=y
CONFIG_FS_FATFS_MOUNT_MKFS=y
CONFIG_FS_FATFS_EXFAT=y
CONFIG_PRINTK=y
CONFIG_MAIN_STACK_SIZE=2048
```

```
CONFIG_PRINTK=y
CONFIG_LOG=y
#CONFIG_LOG_DEFAULT_LEVEL=4
CONFIG_DEBUG_OPTIMIZATIONS=y
CONFIG_LOG_PRINTK=y
CONFIG_LOG_MODE_IMMEDIATE=y
```

```
#CONFIG_USE_SEGGER_RTT=y
#CONFIG_RTT_CONSOLE=y
#CONFIG_LOG_BACKEND_RTT=y
CONFIG_LOG_BACKEND_UART=y
CONFIG_LOG_BUFFER_SIZE=4096
```

```
CONFIG_DISK_DRIVER_SDMMC=y
CONFIG_SPI=y
```

```
CONFIG_HEAP_MEM_POOL_SIZE=512
CONFIG_GPIO=y
```

```
#CONFIG_TEST_SPI_WRITE=y
```



Configuration can be
exhaustive!

Performing a Build

First need to create a configuration

- Specify
 - Target board
 - Zephyr configuration
 - **“Devicetree Overlay”**
 - Custom hardware connected to board

Performing a Build

✓ BUILD CONFIGURATION

The application doesn't have any **build configurations**. Create a build configuration to configure language support.

Create Build Configuration

Add Build Configuration

Select [board](#) and configuration options for blinky:

Board

nrf52840dk_nrf52840

▼

Revision [?]

Revision

☒ Nordic boards ☐ All boards

Configuration [?]

Use build system default

▼

Kconfig fragments [?]

No fragments available

Devicetree overlays [?]

No overlays available

Extra CMake arguments [?]

Add argument

Build directory name [?]

build

Optimization level [?]

Use project default

▼

☒ Build after generating configuration [?]

Build Configuration

Performing a Build

The screenshot shows the Zephyr IDE interface. On the left, the 'ACTIONS' panel has 'Build' selected. The main window displays the 'TERMINAL' tab with the following build logs:

```
[134/140] Linking C static library zephyr\kernel\libkernel.a
[135/140] Linking C executable zephyr\zephyr_pre0.elf
[136/140] Generating linker.cmd
[137/140] Generating isr_tables.c, isrList.bin
[138/140] Building C object zephyr/CMakeFiles/zephyr_final.dir/misc/empty_file.c.obj
[139/140] Building C object zephyr/CMakeFiles/zephyr_final.dir/isr_tables.c.obj
[140/140] Linking C executable zephyr\zephyr.elf
```

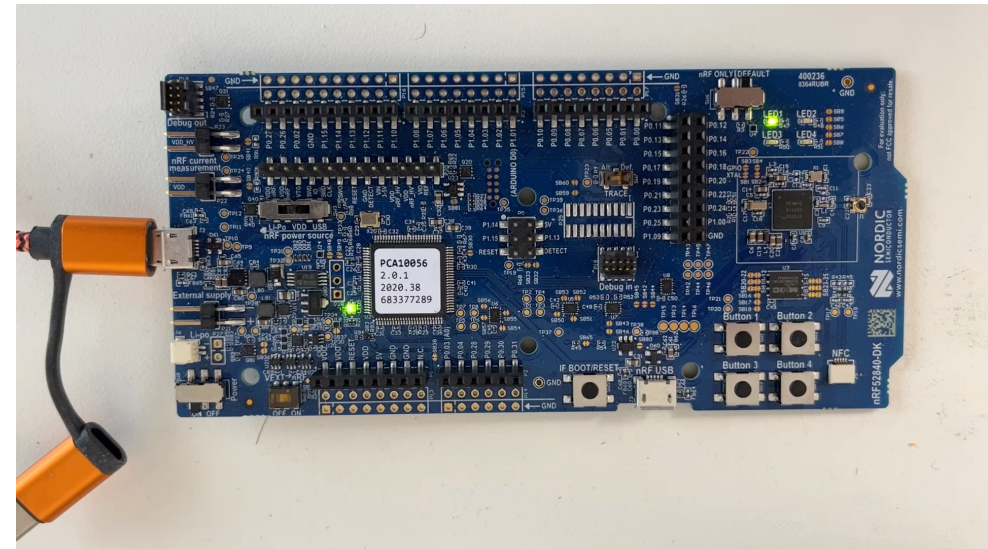
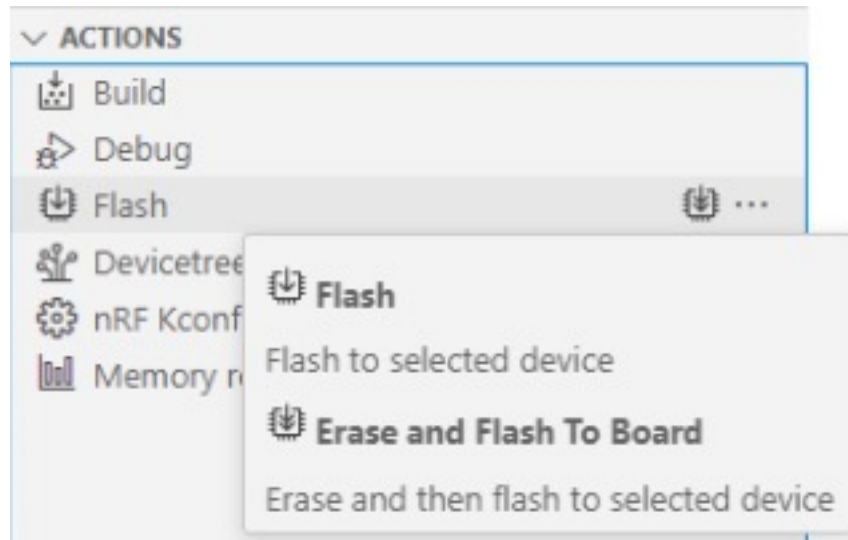
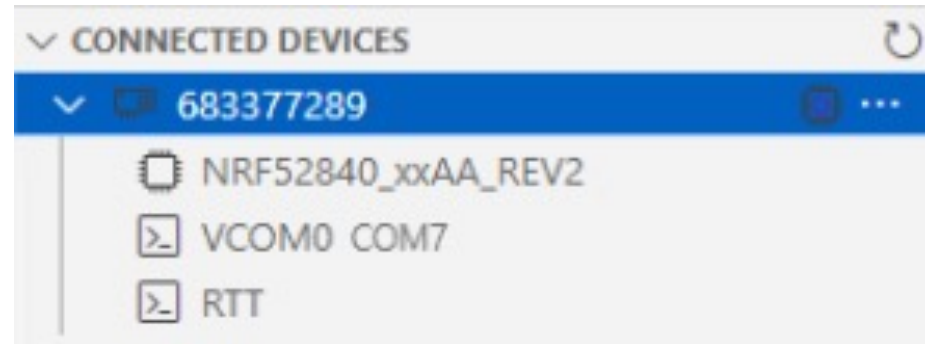
A red box highlights the memory usage table:

Memory region	Used Size	Region Size	%age Used
FLASH:	22096 B	1 MB	2.11%
RAM:	7564 B	256 KB	2.89%
IDT_LIST:	0 GB	2 KB	0.00%

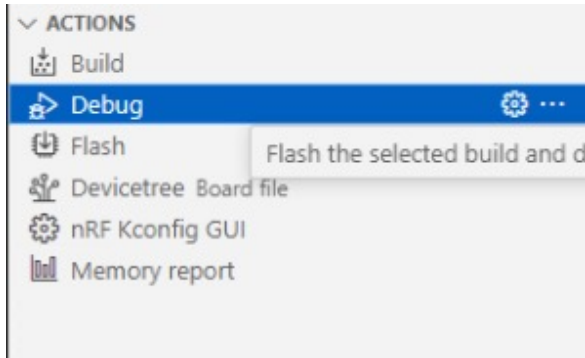
Below the table, a message states: "Terminal will be reused by tasks, press any key to close it."

SUCCESS!!

Flashing and Testing Blinky



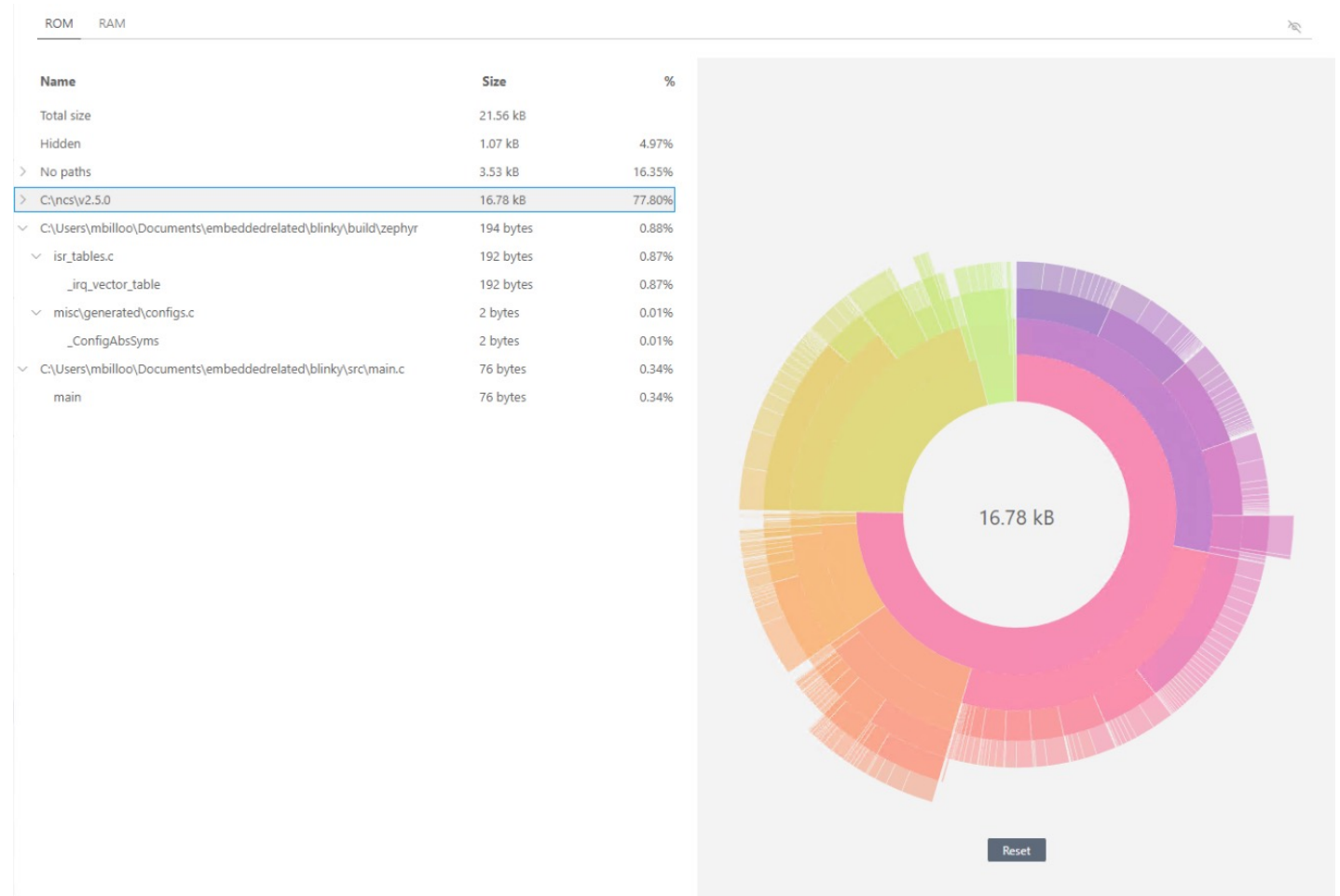
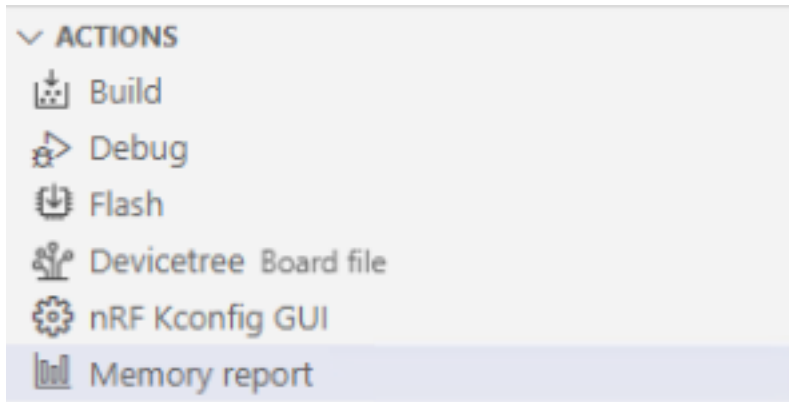
Debugging Blinky



A screenshot of an IDE showing a C program named 'main.c' with a debugger interface. The 'RUN AND DEBUG' tab is active, and the 'Registers' panel shows 'ret: 0'. The code is a simple blink program using GPIO.

```
19  /*
20  static const struct gpio_dt_spec led = GPIO_DT_SPEC_GET(LED0_NODE, gpios);
21
22  int main(void)
23  {
24      int ret;
25
26      if (!gpio_is_ready_dt(&led)) {
27          return 0;
28      }
29
30      ret = gpio_pin_configure_dt(&led, GPIO_OUTPUT_ACTIVE);
31      if (ret < 0) {
32          return 0;
33      }
34
35      while (1) {
36          ret = gpio_pin_toggle_dt(&led);
37          if (ret < 0) {
38              return 0;
39          }
40          k_msleep(SLEEP_TIME_MS);
41      }
42      return 0;
43  }
44
```

Memory Reports





5

“Hello World!” Example



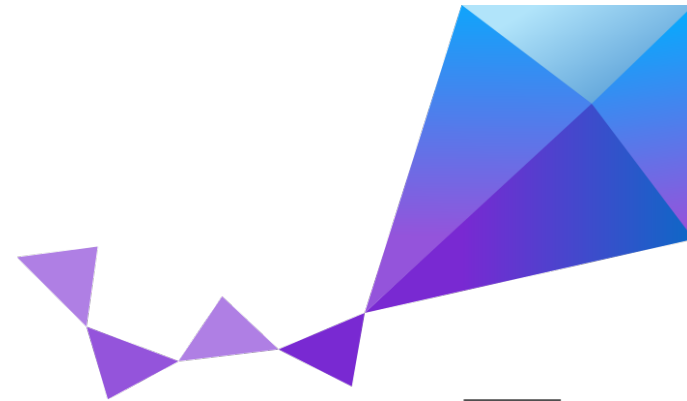
6

Advanced Topics

KConfig

Borrowed from Linux

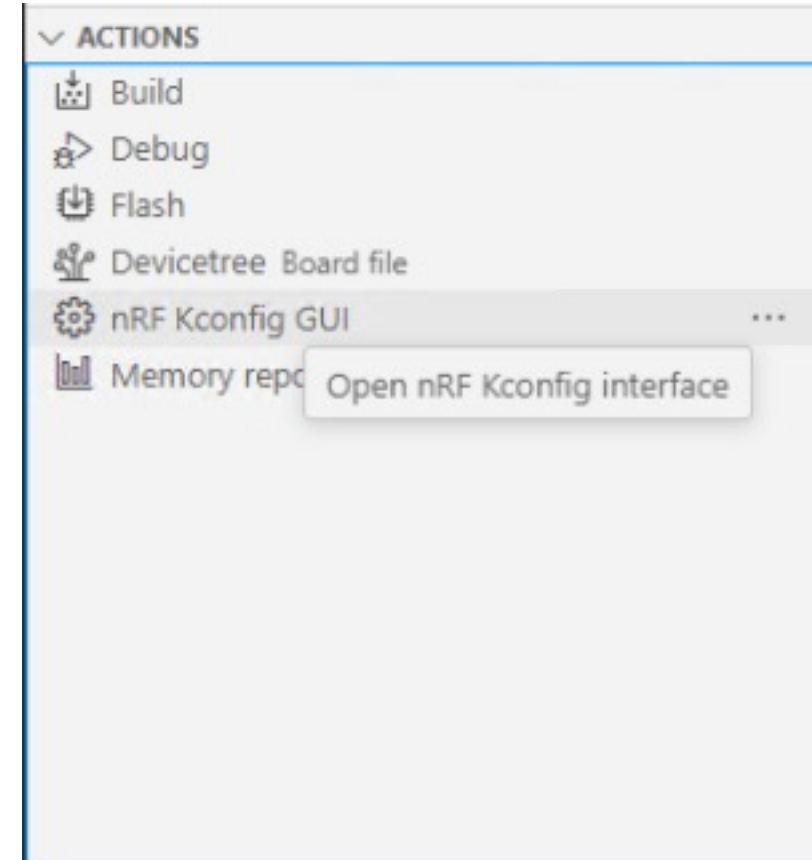
- Used to configure Zephyr features
 - Subsystems
- Similar to specifying macros in other RTOSes
 - Can configure all features in RTOS
 - Not just kernel options
- Configuration
 - Can be file based
 - Usually “prj.conf”
 - Or use GUI



Zephyr™



KConfig



KConfig

← →

Search modules

☰ 🔍

nRF Kconfig

Changes

Apply Save to file Save (minimal)

> Devicetree Info

> Modules

> Board Selection

> Board Options

> SoC/CPU/Configuration Selection

> Hardware Configuration

> ARM Options

> General Architecture Options

> ☒ MPU features

> ☒ Assign appropriate permissions to kernel areas in SRAM

> ☐ Support code/data section relocation

> Floating Point Options

> Cache Options

> General Kernel Options

> Device Options

> Virtual Memory Support

> Device Drivers

> ☐ Require complete C library

> C Library

> C++ Language Support

> ☐ Cyclic redundancy check (CRC) Support

> Additional libraries

> Subsystems and OS Services

> Build and Link Features

> Boot Options

> Compatibility

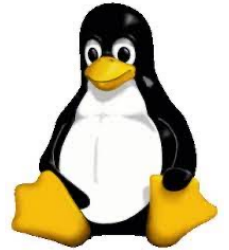
Devicetree

Borrowed from Linux

- Used to configure hardware
 - Pinmux
 - Pinout
 - Board peripherals
- Part of the build process
 - Start with “top-level” device tree file (provided by vendor)
 - Contains references to other device tree files (e.g. pinout configuration)
 - Can customize devicetree using “overlays”
 - Custom hardware



Zephyr™



Devicetree Overlay

```
[app 13:44:35]$ cat sd-spi.overlay
```

```
&spi1 {
    status = "okay";
    pinctrl-0 = <&custom_spi>;
    pinctrl-1 = <&custom_spi>;
    pinctrl-names = "default", "sleep";
    cs-gpios = <&gpio0 2 GPIO_ACTIVE_LOW>;

    sdhc@0: sdhc@0 {
        compatible = "zephyr,sdhc-spi-slot";
        reg = <0>;
        status = "okay";
        label = "SDHC_0";
        mmc {
            compatible = "zephyr,sdmmc-disk";
            status = "okay";
        };
        spi-max-frequency = <24000000>;
    };
};

&pinctrl {
    custom_spi: custom_spi {
        group1 {
            psels = <NRF_PSEL(SPIM_SCK, 0, 26)>,
                <NRF_PSEL(SPIM_MOSI, 0, 27)>,
                <NRF_PSEL(SPIM_MISO, 1, 8)>;
        };
    };
};
```

<https://github.com/mabembedded/zephyr-sd-spi>

Improving IoT Devices With Device Reliability Engineering

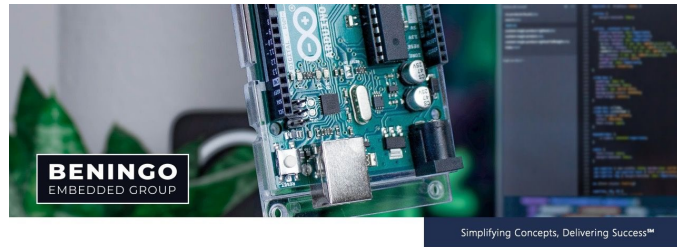
- Device Performance Monitoring
- Device Debugging
- Over-the-Air Updates
- **Built-in to Zephyr!**
- Can try sample under samples/debug/memfault
 - Works with nrf7002 WiFi development kit
 - <https://www.nordicsemi.com/Products/Development-hardware/nRF7002-DK>

7

Going Further

Going Further

Interested in going deeper into Zephyr and RTOS development?
Here are some additional resources you may find helpful:



INTRODUCTION TO ZEPHYR RTOS

Register for \$195 (\$100 off) with coupon
code: **ZEPHYR_WEB23**

Live sessions start January 18, 2024

<https://beningo.mykajabi.com/offers/xeJBY7k5/checkout>

**EMBEDDED SYSTEMS, IOT & DSP
CONFERENCES COMBO FOR \$115!**

Embedded Online Conference 2024	+	IoT Online Conference 2023	+	DSP Online Conference 2023	\$670

Only **\$115** PROMOTIONAL CODE **CYBERWEEK**

Expires December 3

EMBEDDEDONLINECONFERENCE.COM

THANK YOU

Embedded
Online
Conference

w w w . e m b e d d e d o n l i n e c o n f e r e n c e . c o m